

Enterprise Readiness Review Checklist

Universe 3 — Reinsurance Treaty Pricing & Modelling Tool

REPOSITORY	darchville-analytics/modelling_tool
STACK	React 18 + Vite · Express 5 · PostgreSQL · ~250 server and ~400 client modules
AUDIENCE	The software engineer performing the review
SCOPE	Security and login · authorization · architecture · data · configuration · supply chain · observability · resilience · performance · testing · compliance
CONTENTS	131 review tasks across 16 areas, plus 12 findings carried forward from the August 2026 audit
PREPARED	23 August 2026

How to use this checklist. Work top to bottom. Section 0 exists so that nothing below it is reviewed from reading alone — every finding should be demonstrable and every fix provable. Tick an item only when the *Done when* condition holds and you can point at the evidence: a test, a log line, a captured response, or a dated written decision.

Priorities. **P1** blocks an enterprise deployment — do not sign off with one open. **P2** should be closed before or shortly after go-live. **P3** is hygiene and roadmap.

Accepting risk is a valid outcome. Several items ask for a decision rather than a change. A written, dated, signed-off acceptance closes the item; silence does not.

Start with Appendix A. Twelve findings from the August 2026 audit are reproduced at the back with file references. Confirm their current status before spending time anywhere else — A1 is a remotely exploitable path to the highest-privilege account and is enabled in the production manifest.

0 Set Up And Reproduce The Baseline

Do not review from reading alone. Get every gate running locally first, so that every finding you raise later can be demonstrated, and every fix you propose can be proven.

☐	0.1 P1	Run the full verification gate end to end Look at <code>npm run verify</code> (lint + typecheck + frontend budget + server tests + client build + client coverage) Done when The whole gate passes on a clean checkout of main. Record wall-clock time and any flaky test.
☐	0.2 P1	Run the DB-backed integration suite Look at <code>scripts/test-db.sh</code> ; <code>server/tests/integration/*</code> ; CI's integration job in <code>.github/workflows/ci.yml</code> Done when Integration tests run with <code>TEST_WITH_DB=1</code> and the coverage thresholds in <code>server/vitest.config.js</code> are met.
☐	0.3 P2	Stand the stack up from the container manifests Look at <code>docker compose up --build</code> ; <code>Dockerfile</code> ; <code>Dockerfile.jobs</code> ; <code>docker-compose.yml</code> Done when App and Postgres come up, migrations apply, <code>/api/health/deep</code> returns <code>status: ok</code> .
☐	0.4 P2	Diff the deployed configuration against the sample Look at <code>render.yaml</code> vs <code>.env.example</code> (14 KB of documented flags); <code>server/src/config/env.js</code> Done when You have a written list of every env var that production sets, omits, or overrides — this is where most findings live.
☐	0.5 P2	Read the prior audits before duplicating work Look at <code>docs/CODEBASE_AUDIT_2026-08.md</code> , <code>docs/CODEBASE_AUDIT_2026-07.md</code> , <code>SECURITY.md</code> Done when Each prior finding is marked verified-fixed, still-open, or regressed. See Appendix A.
☐	0.6 P3	Capture a code-size and hot-spot baseline Look at <code>find client/src server/src -name '*.js*' xargs wc -l sort -rn head -30</code> Done when You can name the ten largest modules and why each is large — they set the refactor agenda.

1 Authentication And Login

The login path is the single highest-value surface in the system. Trace every way a session can be created, and prove that no path mints one without a verified credential.

☐	1.1 P1	Enumerate every path that can create a session Look at <code>server/src/routes/auth.js</code> (POST <code>/auth/login</code> , <code>/auth/name-login</code> , POST <code>/auth/users</code>), <code>server/src/routes/sso.js</code> , <code>server/src/services/sessions.js</code> Done when You have a complete, written list of session-minting entry points. Any path that does not verify a secret is a blocker.
☐	1.2 P1	Audit the passwordless name-login pilot flag Look at <code>ALLOW_NAME_AUTH</code> in <code>render.yaml</code> ; the <code>/auth/name-login</code> handler in <code>server/src/routes/auth.js</code> Done when Either the flag is off in production, or a boot-time fence in <code>config/env.js</code> refuses to start with it on in production — mirroring <code>checkDemoAuthConfig</code> . See A1.
☐	1.3 P1	Audit the demo-auth backdoor fence Look at <code>checkDemoAuthConfig</code> in <code>server/src/config/env.js</code> ; <code>ALLOW_DEMO_AUTH</code> reads in <code>middleware/requestContext.js</code> Done when <code>ALLOW_DEMO_AUTH=true</code> with <code>NODE_ENV=production</code> is a fatal boot error, and the <code>demo2026</code> shortcut is unreachable in production. Confirm by test, not by comment.
☐	1.4 P1	Confirm no route trusts <code>x-user-*</code> headers for authorization Look at <code>demoUserFromHeaders</code> in <code>server/src/middleware/requestContext.js</code> ; grep the whole server for <code>x-user-</code> Done when Header identity is reachable only under <code>ALLOW_DEMO_AUTH</code> ; every other read of those headers is logging-only.
☐	1.5 P1	Review the public (unauthenticated) endpoint allow-list Look at The <code>PUBLIC_AUTH</code> set in <code>server/src/routes/auth.js</code> Done when Every entry is justified in writing. GET <code>/auth/users</code> returning the full roster with role codes to anonymous callers is an enumeration finding — see A2.

☐	1.6 P1	Verify password hashing parameters and rehash-on-login Look at <code>server/src/lib/passwordHash.js</code> , <code>PASSWORD_SCRIPT_COST</code> ; <code>needsRehash</code> usage in <code>routes/auth.js</code> Done when <code>script</code> is <code>async</code> (off the event loop), <code>cost</code> is tuned to the production instance size, and logins transparently upgrade legacy hashes.
☐	1.7 P1	Test the password policy at every place a password is set Look at <code>validatePasswordStrength</code> in <code>server/src/routes/auth.js</code> ; <code>change-password</code> , <code>admin-create</code> , <code>self-register</code> , <code>SSO provisioning</code> Done when One shared policy gates all paths. No path can set a password that another path would reject.
☐	1.8 P1	Hunt for seeded or shared credentials in migrations and seeds Look at <code>server/src/db/migrations/132_saudi_re_demo_personas.sql</code> ; <code>server/src/db/seeds/</code> ; <code>git log -S password</code> Done when No migration that runs in production installs a known password or clears <code>must_change_password</code> . See A3.
☐	1.9 P1	Prove brute-force controls actually engage Look at <code>Login limiter</code> in <code>server/src/app.js</code> (IP + submitted identity, 5 per 15 min); <code>failed_attempts / locked_until</code> in <code>routes/auth.js</code> Done when You have driven a lockout and a rate-limit rejection by hand. Note that neither engages on a login path that never fails.
☐	1.10 P1	Check limiter counting under multiple instances Look at <code>server/src/lib/rateLimitStore.js</code> ; <code>REDIS_URL</code> ; <code>rateLimitStoreEnabled()</code> in <code>startup/productionPosture.js</code> Done when Either a shared Redis store is configured, or the deployment is provably single-instance and that is a documented constraint.
☐	1.11 P2	Verify limiter bypass flags cannot exist in production Look at <code>LOAD_TEST / ALLOW_DEMO_AUTH</code> bypass in <code>server/src/app.js</code> (~line 173) Done when The bypass is fenced by the same production check as the auth backdoor, not only by a warning log.
☐	1.12 P1	Review the SSO/OIDC integration end to end Look at <code>server/src/routes/sso.js</code> , <code>services/identity/oidcClient.js</code> , <code>provisioning.js</code> , <code>backchannelLogout.js</code> , <code>lib/ssoStateCookie.js</code> Done when State and nonce are validated, the ID token signature and issuer/audience are checked, and the redirect URI is allow-listed — no open redirect.
☐	1.13 P1	Verify MFA is enforced by the application, not assumed from the IdP Look at <code>IDENTITY_REQUIRED_ACR / IDENTITY_REQUIRED_AMR / IDENTITY_ENFORCE_MFA</code> in <code>server/src/config/identity.js</code> Done when With <code>IDENTITY_ENFORCE_MFA</code> on, an SSO config lacking <code>acr/amr</code> requirements is a fatal boot error, and a token missing the required claim is rejected.
☐	1.14 P1	Review claim-to-role mapping for privilege escalation Look at <code>mapClaimsToRole</code> in <code>server/src/config/identity.js</code> ; <code>IDENTITY_ROLE_MAP</code> , <code>IDENTITY_DEFAULT_ROLE</code> Done when An unknown, missing, or attacker-influenced claim can only ever yield the lowest-privilege role. Write a test for the adversarial claim shapes.
☐	1.15 P2	Review the break-glass local-admin path Look at <code>isBreakGlassUser / IDENTITY_BREAK_GLASS_USERS</code> in <code>server/src/config/identity.js</code> ; <code>docs/runbooks/sso-mfa-break-glass.md</code> Done when The list is short, named, audited on use, alerted on, and reviewed on a stated cadence.
☐	1.16 P2	Check open self-registration is off and cannot self-elevate Look at <code>ALLOW_OPEN_REGISTRATION</code> in <code>render.yaml</code> ; the <code>POST /auth/users self-gate</code> in <code>routes/auth.js</code> Done when Registration is off in production; when on, a new account cannot choose its own role or hierarchy level.
☐	1.17 P2	Confirm the forced-password-change gate cannot be walked around Look at <code>server/src/app.passwordGate.test.js</code> ; <code>mustChangePassword</code> in <code>middleware/requestContext.js</code> Done when A <code>must_change_password</code> user is blocked from all mutations, not merely redirected by the client.

☐	1.18 P2	Review login-failure responses for user enumeration Look at Error bodies and timing in the <code>/auth/login</code> handler Done when Unknown-user and wrong-password return the same status, body, and broadly the same latency.
☐	1.19 P2	Confirm security-relevant auth events are alerted, not just logged Look at <code>server/src/services/securityAlerts.js</code> ; <code>emitSecurityAlert</code> call sites Done when Lockouts, break-glass logins, mass revocations and posture errors reach a channel a human actually watches.

2 Sessions, Tokens And Cookies

A correct login is worthless if the resulting session cannot be revoked, or can be replayed.

☐	2.1 P1	Verify token signing, algorithm and secret strength Look at <code>server/src/lib/authToken.js</code> ; the <code>validateEnv()</code> secret gate in <code>server/src/config/env.js</code> Done when The algorithm is pinned (no none/alg confusion), the secret is ≥ 32 chars, and boot fails without it in production.
☐	2.2 P1	Confirm role and mandate are re-read per request, not carried in the token Look at <code>loadUserAndSession</code> in <code>server/src/middleware/requestContext.js</code> ; the <code>v_user_mandate</code> view Done when A demotion, deactivation or COB restriction takes effect on the very next request with an existing token.
☐	2.3 P1	Test session revocation and the mass-revoke epoch Look at <code>server/src/services/sessions.js</code> (<code>revokeSession</code> , <code>revokeAllForUser</code>); <code>uw_user.session_epoch</code> Done when Logout, password change, role change and deactivation each invalidate live sessions immediately. Prove it with a held cookie.
☐	2.4 P1	Review cookie attributes Look at <code>server/src/lib/authCookies.js</code> (<code>AUTH_COOKIE</code> , <code>CSRF_COOKIE</code>) Done when <code>HttpOnly</code> , <code>Secure</code> , <code>SameSite</code> , <code>Path</code> and <code>domain</code> are correct for production, and the CSRF cookie is deliberately readable while the auth cookie is not.
☐	2.5 P1	Test CSRF double-submit protection on every mutating route Look at <code>server/src/lib/csrf.js</code> ; <code>csrfProtection</code> wiring in <code>app.js</code> ; <code>CSRF_HEADER</code> Done when A cookie-authenticated POST/PUT/PATCH/DELETE without a matching header token is rejected. Check for routes mounted outside the guard.
☐	2.6 P2	Review session lifetime and idle handling Look at <code>SESSION_TTL_HOURS</code> (8 in <code>render.yaml</code>); <code>auth_session.expires_at</code> Done when Absolute and idle lifetimes are deliberate and documented; expiry is enforced server-side, not by cookie max-age alone.
☐	2.7 P2	Check OIDC back-channel logout closes local sessions Look at <code>server/src/services/identity/backchannelLogout.js</code> Done when An IdP-initiated logout revokes the matching local <code>auth_session</code> rows; the endpoint verifies the logout token.
☐	2.8 P2	Confirm the session table is bounded Look at <code>auth_session</code> schema and any cleanup job; <code>server/src/db/maintenance/</code> Done when Expired and revoked sessions are purged on a schedule; the table cannot grow without limit.

3 Authorization, Roles And Mandates

Six role codes (CE, CU, TD, TM, TUW/UW) and a hierarchy level drive edit rights, approvals and underwriting limits. Authorization must be enforced server-side on every route.

☐	3.1 P1	Map every route to its guard Look at <code>server/src/routes/registerApiRoutes.js</code> ; <code>requireAuth</code> , <code>requireMinLevel</code> in <code>middleware/requestContext.js</code> Done when You have a route-by-route table showing the auth and role guard on each. Every unguarded route is justified in writing.
--------------------------	-------------------------	--

☐	3.2 P1	Verify the global mutation guard cannot be bypassed Look at <code>guardApiMutations</code> in <code>server/src/services/permissions.js</code> ; its mount order in <code>app.js</code> Done when Routers mounted before or outside the guard are identified; none permits a write.
☐	3.3 P1	Test the ownership / edit-lock rule adversarially Look at <code>computeEditPermission</code> and <code>assertCanEdit</code> in <code>server/src/services/permissions.js</code> Done when Only the current assignee may mutate a contract, quote or fac risk; hierarchy level does not silently grant edit. Attempt cross-user writes by ID.
☐	3.4 P1	Probe for insecure direct object references Look at Any route taking <code>:id</code> — <code>routes/treaties.js</code> , <code>quotes.js</code> , <code>facultative.js</code> , <code>dashboard.js</code> , <code>workbench.js</code> Done when Fetching another user's or another entity's ID either 403s or filters by mandate; it never leaks by ID guessing.
☐	3.5 P1	Verify approval authority and underwriting limits are server-enforced Look at <code>server/src/services/approvals.js</code> , <code>workflow.js</code> , <code>quoteWorkflow.js</code> , <code>quoteBind.js</code> ; <code>effective_limit_usd</code> Done when A user cannot approve or bind above their mandate limit even with a hand-crafted request. The client's own gating is not the control.
☐	3.6 P2	Check class-of-business and treaty-type restrictions are applied in queries Look at <code>restricted_cob_ids</code> , <code>treaty_type_scope</code> from <code>v_user_mandate</code> ; their use in repositories Done when Restrictions are applied in the SQL predicate, not filtered in the client or after serialization.
☐	3.7 P2	Review the state machine for illegal transitions Look at <code>server/src/lib/statusMachine.js</code> ; <code>services/workflowTransition.test.js</code> ; <code>docs/pricing-signoff-required.md</code> Done when Every transition is validated server-side; SIGNED / DECLINED / NTU are terminal; sign-off cannot be skipped.
☐	3.8 P2	Confirm admin and reference-data routes are level-gated Look at <code>server/src/routes/facultativeReference.js</code> ; <code>services/facReferenceAdminService.js</code> ; <code>client/src/screens/admin</code> Done when Rate tables and reference data can only be changed by the intended roles, and every change is audited and versioned.

4 Application Security Controls

Headers, input, output, uploads and outbound calls.

☐	4.1 P1	Review the Content-Security-Policy and its nonce plumbing Look at <code>server/src/app.js</code> (helmet config, <code>res.locals.cspNonce</code> , <code>serveIndex</code>); <code>server/src/app.csp.test.js</code> ; <code>SECURITY.md</code> Done when The policy is enforcing (not report-only), has no <code>unsafe-inline/unsafe-eval</code> for scripts, and the nonce is per-request and unpredictable.
☐	4.2 P1	Review CORS for production Look at <code>createCorsOptions</code> in <code>server/src/app.js</code> ; <code>CORS_ORIGIN</code> in <code>render.yaml</code> Done when Production uses an explicit allow-list; origin reflection with credentials is impossible in production. A <code>*</code> value is a finding.
☐	4.3 P1	Verify input validation coverage across the API Look at <code>server/src/validation/*</code> (zod schemas); <code>server/src/lib/validate.js</code> Done when Every route with a body or query parameter has a schema. List the routes that do not — they are the injection and mass-assignment surface.
☐	4.4 P1	Check for SQL injection and unsafe identifier interpolation Look at Grep for template literals inside <code>pool.query</code> ; note the interpolated table names in <code>services/permissions.js</code> Done when All user-supplied values are parameterized; any interpolated identifier comes from a fixed internal allow-list, never from a request.
☐	4.5 P1	Review file upload handling Look at <code>server/src/lib/uploadValidation.js</code> , <code>uploadStorage.js</code> ; multer config; <code>UPLOAD_DIR</code> Done when Type and size are validated by content and not by filename alone; stored names are non-guessable; path traversal is impossible; downloads are access-checked.

☐	4.6 P1	Confirm durable upload storage in production Look at <code>remoteStorageConfigured()</code> / <code>localUploadsAllowed()</code> in <code>lib/uploadStorage.js</code> ; Cloudinary vars in <code>render.yaml</code> Done when Either object storage is configured, or ephemeral local disk is an explicit, documented acceptance. As shipped this errors at boot — see A4.
☐	4.7 P1	Stop internal error detail reaching clients Look at <code>server/src/middleware/errorHandler.js</code> (~line 47) Done when For status ≥ 500 in production the client gets a generic message plus a request ID; the detail stays in the server log. See A5.
☐	4.8 P2	Review outbound HTTP clients for SSRF and secret leakage Look at <code>server/src/lib/axcoClient.js</code> , <code>googlePlaces.js</code> , <code>imfClient.js</code> , <code>worldBankClient.js</code> , <code>llmClient.js</code> , <code>facDocAi.js</code> Done when Destinations are fixed or allow-listed, timeouts and retry caps are set, API keys never reach logs or client responses.
☐	4.9 P1	Review the AI/LLM surface for prompt injection and data egress Look at <code>server/src/lib/llmClient.js</code> , <code>aiGovernance.js</code> , <code>facDocAiPrompts.js</code> ; <code>routes/ai.js</code> , <code>aiMarket.js</code> , <code>aiCedant.js</code> ; <code>services/renewalPack/extractor.js</code> Done when Untrusted document content cannot alter instructions or trigger tool/DB actions; what leaves the estate is documented and approved; AI routes are authenticated and rate-limited.
☐	4.10 P2	Verify request timeouts and body-size limits Look at <code>server/src/middleware/requestTimeout.js</code> ; the <code>express.json</code> limit in <code>app.js</code> Done when Slow and oversized requests are rejected with a clear code; limits are set for every parser including multipart.
☐	4.11 P2	Check caching headers cannot leak per-user data Look at <code>server/src/middleware/httpCache.js</code> ; <code>cacheStats()</code> Done when No authenticated, user-scoped response is publicly cacheable; the cache key includes identity wherever the payload is mandate-filtered.
☐	4.12 P2	Review the container image posture Look at <code>Dockerfile</code> , <code>.dockerignore</code> ; <code>.github/workflows/image-security.yml</code> (Syft SBOM + Trivy) Done when Non-root user, pinned base digest, no secrets in layers, no dev dependencies in the runtime image, and the Trivy gate actually fails the build.
☐	4.13 P2	Confirm secret scanning and SAST are running and enforcing Look at <code>.github/workflows/codeql.yml</code> ; <code>SECURITY.md</code> (Static analysis & secret scanning) Done when CodeQL runs on PRs and is required; the workflow's soft-fail path is not silently hiding that scanning is disabled.
☐	4.14 P2	Verify database transport security is enforced, not assumed Look at <code>server/src/db/pool.js</code> ; <code>DATABASE_URL</code> in <code>render.yaml</code> ; <code>DEPLOYMENT.md</code> Done when An explicit <code>ssl</code> option is set in production, or boot asserts <code>sslmode=require</code> in the connection string. See A6.

5 Architecture And Code Structure

Roughly 250 server modules and 400 client modules. Judge the boundaries, not the line count.

☐	5.1 P1	Validate the documented architecture against the code Look at <code>docs/architecture.md</code> (core model, quote/treaty duality, save pattern, guardrails) Done when The document describes the system as it is today. Where it does not, either the code or the document is a finding.
☐	5.2 P1	Assess layering discipline: routes, services, repositories Look at <code>server/src/routes/</code> vs <code>services/</code> vs <code>modules/pricing/repositories/</code> Done when Routes stay thin; business rules live in services; SQL lives in repositories. Record every route with inline SQL or business logic.

☐	5.3 P1	Decide where the authoritative pricing calculation lives Look at <code>client/src/utils/npPricingEngine.js</code> (983 lines), <code>shared/pricingMath.js</code> , <code>server/src/lib/pricingVerifier.js</code> , <code>PRICING_STRICT</code> Done when A written decision: keep client-side computation with server drift rejection, or move the primary calculation server-side. The financial authority must not sit in a tamperable client. See A9.
☐	5.4 P2	Review the module boundaries of the newer modules/ code Look at <code>server/src/modules/pricing/{controllers,services,repositories}</code> , <code>modules/gemVulnerability</code> , <code>modules/hazusVulnerability</code> Done when The modular pattern is either the target for the whole server or explicitly scoped; two competing structures need a documented migration path.
☐	5.5 P2	Identify god-modules and set a refactor agenda Look at <code>client/src/components/cedant/CedantSummaryTabs.jsx</code> (2,066 lines), <code>MarketIntelligenceModal.jsx</code> (1,212), <code>server/src/routes/auth.js</code> (1,144) Done when Each oversized module has an owner, a stated risk, and a split plan — or a documented reason to leave it alone.
☐	5.6 P2	Confirm shared logic is genuinely shared Look at <code>shared/pricingMath.js</code> , <code>shared/onLevel.js</code> , <code>shared/fac/</code> Done when No formula is implemented twice with room to drift. Any duplicate is a correctness risk, not a style issue.
☐	5.7 P2	Review concurrency and optimistic locking coverage Look at <code>server/src/db/optimisticLock.js</code> , <code>withTransaction.js</code> ; <code>docs/optimistic-locking-coverage.md</code> ; <code>client/src/hooks/useEditLock.js</code> Done when Every multi-step write is transactional; concurrent edits collide loudly rather than silently overwriting.
☐	5.8 P2	Review the background-job architecture Look at <code>server/src/jobs/refreshLdfBenchmarks.js</code> , <code>refreshFacAccumulation.js</code> ; <code>.github/workflows/scheduled-jobs.yml</code> ; <code>Render Cron</code> in <code>render.yaml</code> ; <code>Dockerfile.jobs</code> Done when Jobs are idempotent, safe to run concurrently, alert on failure, and are not silently skipped when a deploy fails.
☐	5.9 P3	Make the tenancy decision explicit Look at <code>No tenant_id</code> and no row-level security anywhere in <code>server/src/db/migrations/</code> Done when <code>docs/architecture.md</code> states single-tenant deploy-per-customer as a deliberate choice, with the cost of changing it later.
☐	5.10 P2	Assess API contract stability and versioning Look at <code>server/src/routes/registerApiRoutes.js</code> ; client fetch layer in <code>client/src/hooks/useResource.ts</code> Done when There is a stated policy for breaking changes to <code>/api</code> — versioning, deprecation window, or a documented tight-coupling acceptance.

6 Data Layer, Migrations And Integrity

140 forward-only migrations against Postgres. The schema is the asset; treat migration safety as a first-class concern.

☐	6.1 P1	Review the migration runner for safety Look at <code>server/src/startup/runMigrations.js</code> , <code>server/src/db/migrate.js</code> ; <code>RUN_MIGRATIONS_ON_BOOT</code> ; <code>preDeployCommand</code> in <code>render.yaml</code> Done when Migrations are advisory-locked, transactional, and applied exactly once across concurrent instances.
☐	6.2 P1	State the rollback policy explicitly Look at 140 migrations in <code>server/src/db/migrations/</code> with no down scripts; <code>docs/runbooks/disaster-recovery.md</code> Done when Forward-only is written down as policy, with restore-from-backup as the stated recovery path and a rehearsed timing. See A10.
☐	6.3 P1	Check for long-locking DDL in recent migrations Look at <code>docs/migration-audit.md</code> ; <code>docs/runbooks/validate-not-valid-fks.md</code> ; recent files 133–140 Done when Index creation is concurrent, constraints are added <code>NOT VALID</code> then validated, and no migration takes a long exclusive lock on a hot table.

☐	6.4 P1	Verify referential integrity and constraints exist in the database Look at <code>docs/database-design-audit.md</code> ; foreign keys, <code>NOT NULL</code> , <code>CHECK</code> constraints across the schema Done when Invariants are enforced by the database, not only by application code. List the ones that are application-only.
☐	6.5 P2	Review connection-pool sizing against the deployment Look at <code>server/src/db/pool.js</code> , <code>poolWatchdog.js</code> ; <code>DB_POOL_MAX/DB_POOL_MIN</code> ; <code>docs/scaling.md</code> Done when <code>instances × DB_POOL_MAX</code> fits comfortably inside the database's <code>max_connections</code> , with headroom for jobs and migrations.
☐	6.6 P2	Review numeric types used for money and rates Look at Column types across the pricing tables; rounding in <code>shared/pricingMath.js</code> Done when Monetary values use exact numeric types with a stated rounding convention; no float drift reaches a bound contract.
☐	6.7 P2	Confirm N+1 queries and unbounded result sets are controlled Look at <code>server/src/db/contractJoins.js</code> , <code>batchInsert.js</code> ; pagination headers <code>X-Total-Count/X-Page</code> Done when List endpoints paginate by default; no request can ask the database for an unbounded scan.
☐	6.8 P2	Verify retention jobs run and are proven Look at <code>server/scripts/cleanup-snapshots.js</code> (30-day <code>import_snapshots</code> window); <code>docs/data-retention.md</code> Done when The nightly job is scheduled, monitored, and its last successful run is observable.
☐	6.9 P3	Review materialized-view refresh strategy Look at <code>server/src/jobs/refreshLdfBenchmarks.js</code> ; <code>mv_ldf_benchmark_*</code> Done when Refreshes are concurrent where possible, cannot pile up, and stale views are detectable.

7 Secrets And Configuration

Most findings in a mature codebase live in configuration rather than code.

☐	7.1 P1	Scan the full git history for committed secrets Look at <code>git log -p grep -iE 'secret api[_-]?key password token'</code> ; GitHub secret scanning Done when No live credential exists in any commit. Anything found is rotated first, then removed from history.
☐	7.2 P1	Verify fail-fast validation of required production config Look at <code>validateEnv()</code> / <code>validateRuntimeEnv()</code> in <code>server/src/config/env.js</code> ; <code>startup/bootstrap.js</code> Done when A missing or weak <code>AUTH_TOKEN_SECRET</code> , or a missing <code>DATABASE_URL</code> , refuses to boot rather than degrading quietly.
☐	7.3 P1	Review every boot-time posture warning as a decision Look at <code>server/src/startup/productionPosture.js</code> (identity, uploads, CORS, Redis) Done when Each warning or error the current production config produces is either fixed or has a signed-off, written acceptance.
☐	7.4 P2	Confirm a secret rotation procedure exists and is rehearsed Look at <code>AUTH_TOKEN_SECRET</code> , <code>DATABASE_URL</code> , Cloudinary, LLM and third-party keys; <code>docs/runbooks/incident-response.md</code> Done when There is a documented, timed procedure for rotating each secret, including what it invalidates (rotating the token secret logs everyone out).
☐	7.5 P2	Check environment parity Look at <code>.env.example</code> vs <code>render.yaml</code> vs <code>docker-compose.yml</code> vs <code>ecosystem.config.cjs</code> Done when Every flag that changes security behaviour is set explicitly in production rather than relying on a default.
☐	7.6 P2	Verify feature flags are enumerated with owners and expiry Look at <code>PRICING_STRICT</code> , <code>ALLOW_NAME_AUTH</code> , <code>ALLOW_DEMO_AUTH</code> , <code>ALLOW_OPEN_REGISTRATION</code> , <code>ALLOW_LOCAL_UPLOADS</code> , <code>OTEL_ENABLED</code> , <code>LOAD_TEST</code> Done when A single table lists each flag, its production value, its owner, and its removal date. Temporary pilot flags have a hard end date.

8 Dependencies And Supply Chain

Two workspaces plus a root, with overrides already in use. Keep the runtime clean and the gate honest.

☐	8.1 P1	Triage every outstanding advisory Look at <code>npm audit</code> at root, <code>--prefix server</code> , <code>--prefix client</code> ; CI's prod dependency audit job Done when Runtime dependencies are clean at the CI threshold. Dev-only advisories (the current root highs are build-chain: brace-expansion, postcss, nanoid, shell-quote, ip-address) are listed with a rationale.
☐	8.2 P2	Review the pinning and override strategy Look at overrides in root <code>package.json</code> and <code>server/package.json</code> ; <code>package-lock.json</code> committed Done when Overrides exist to close advisories, are dated, and have a removal condition once upstream ships a fix.
☐	8.3 P2	Verify the licence scan covers what ships Look at CI dependency license scan job in <code>.github/workflows/ci.yml</code> Done when Production dependencies of both workspaces are scanned and the allow-list is deliberate for a commercial, unlicensed product.
☐	8.4 P2	Confirm the SBOM is produced and retained Look at <code>.github/workflows/image-security.yml</code> (Syft) Done when An SBOM is generated per image build and retrievable for any deployed version.
☐	8.5 P2	Check the Node runtime is pinned and supported Look at <code>.nvmrc</code> , <code>.node-version</code> , engines in <code>package.json</code> , the Dockerfile base image Done when All four agree, and the version is on a supported LTS line for the expected life of the release.
☐	8.6 P3	Review automated dependency updates Look at Dependabot/Renovate configuration in <code>.github/</code> Done when Updates arrive on a schedule with CI gating, rather than accumulating until a forced upgrade.

9 Observability And Operations

You cannot run an enterprise system you cannot see.

☐	9.1 P1	Verify tracing and metrics are enabled in production Look at <code>OTEL_ENABLED</code> and the rest of the OTEL vars; <code>server/src/observability/telemetry.js</code> , <code>otelInit.js</code> , <code>httpMetrics.js</code> , <code>poolMetrics.js</code> ; <code>docs/observability.md</code> Done when Traces and metrics reach a real backend from production, and someone has opened the dashboard this month.
☐	9.2 P1	Add log-level control Look at <code>server/src/lib/logger.js</code> — no <code>LOG_LEVEL</code> threshold today Done when Verbosity is controllable by environment variable so an incident can be turned up and cost turned down. See A8.
☐	9.3 P1	Confirm logs never contain credentials or personal data Look at <code>logger.js</code> redaction; log lines in <code>routes/auth.js</code> , <code>middleware/requestContext.js</code> , the outbound clients Done when Passwords, tokens, cookies, and API keys are impossible to log. Verify by grepping a real log sample, not by reading the redaction list.
☐	9.4 P2	Verify request correlation end to end Look at <code>server/src/middleware/requestId.js</code> ; <code>X-Request-Id</code> in <code>EXPOSED_HEADERS</code> ; client error surfacing Done when A user-visible error carries an ID that finds the exact server log line and trace.
☐	9.5 P1	Check health endpoints are wired to the platform Look at <code>/api/health</code> , <code>/api/health/deep</code> in <code>app.js</code> ; <code>healthCheckPath</code> in <code>render.yaml</code> Done when The load balancer uses the shallow check; the deep check is monitored separately and its <code>X-Pool-Waiting</code> value is alerted on.
☐	9.6 P1	Define alerting and on-call Look at <code>docs/runbooks/incident-response.md</code> ; <code>services/securityAlerts.js</code> Done when There is a named alert for error rate, latency, pool saturation, job failure and boot-posture errors, each routed to someone on call.

☐	9.7 P2	Review business-level metrics Look at <code>server/src/observability/businessMetrics.js</code> , <code>aiMetrics.js</code> Done when Quote, bind and pricing-drift volumes are observable, so a functional regression is visible before a customer reports it.
☐	9.8 P2	Confirm graceful shutdown behaves under real deploys Look at <code>server/src/startup/gracefulShutdown.js</code> ; keep-alive tuning in <code>bootstrap.js</code> Done when A SIGTERM during load drains in-flight requests without dropped connections. Prove it under k6.

10 Reliability, Availability And Recovery

Enterprise buyers ask for numbers here, and expect evidence behind them.

☐	10.1 P1	State the availability topology honestly Look at <code>plan: starter</code> , single instance, no <code>numInstances</code> in <code>render.yaml</code> Done when The absence of HA is either fixed (multiple instances behind the load balancer) or documented as an accepted limitation with its downtime implications. See A11.
☐	10.2 P1	Prove backups exist and restores work Look at <code>docs/backup-recovery.md</code> (RPO/RTO commitments); the restore-verification drill in <code>.github/workflows/scheduled-jobs.yml</code> Done when A restore has been performed within the last quarter, timed, and the measured time is inside the committed RTO.
☐	10.3 P1	Rehearse the disaster-recovery runbook Look at <code>docs/runbooks/disaster-recovery.md</code> Done when Someone who did not write the runbook has followed it end to end successfully, and the gaps they hit are fixed.
☐	10.4 P2	Verify scale-out prerequisites before they are needed Look at <code>REDIS_URL</code> for shared rate limits; upload storage; <code>docs/scaling.md</code> ; <code>ecosystem.config.cjs</code> Done when Every per-process assumption is listed, and each has a shared-state answer before a second instance is added.
☐	10.5 P2	Check deploy and rollback procedure Look at <code>deploy.sh</code> , <code>render.yaml</code> (<code>preDeployCommand</code>), <code>DEPLOYMENT.md</code> Done when Rollback is one documented step, and its interaction with forward-only migrations is stated.
☐	10.6 P2	Confirm timeouts and failure handling for third-party outages Look at The outbound clients in <code>server/src/lib/</code> Done when An outage at any third party degrades a feature, never the whole application.

11 Performance And Scale

Measure before optimising, and re-measure after any change made from this checklist.

☐	11.1 P1	Run the load tests and record a capacity baseline Look at <code>npm run loadtest</code> , <code>loadtest:10vu</code> , <code>loadtest:capacity</code> ; <code>load-test/k6/</code> ; <code>loadtest:summarize</code> Done when You have p50/p95/p99 latency and error rate at a stated concurrency, and know the level at which the service degrades.
☐	11.2 P1	Profile the slowest endpoints against real data volumes Look at <code>EXPLAIN ANALYZE</code> on the heaviest queries; <code>docs/scaling.md</code> Done when The top five slowest endpoints are identified with an index or query plan for each. Test against production-scale row counts, not seed data.
☐	11.3 P2	Verify the frontend bundle budget is meaningful Look at <code>npm run budget:frontend</code> , <code>scripts/frontend-budget.mjs</code> ; lazy route loading Done when The budget reflects a real target on a real network, and it fails the build when exceeded.

☐	11.4 P2	Check pool saturation behaviour under load Look at <code>poolWatchdog.js</code> ; <code>X-Pool-Waiting</code> from <code>/api/health/deep</code> Done when Under the capacity test the pool does not queue sustainedly; if it does, <code>DB_POOL_MAX</code> and instance count are re-derived.
☐	11.5 P2	Review heavy computation on the request path Look at Pricing, clustering and export paths — <code>client/src/logic/portfolioClustering.js</code> , <code>services/renewalPackExport/</code> , <code>exceljs</code> use Done when Long-running work is bounded, streamed, or moved off the request path; it cannot block the event loop.

12 Testing And Delivery Pipeline

Around 2,000 server and 1,000 client tests already exist. Judge what they cover, not how many there are.

☐	12.1 P1	Check coverage where the money is Look at <code>server/vitest.config.js</code> thresholds; client coverage floors; <code>client/src/utils/npPricingEngine.js</code> Done when Auth, workflow, pricing and document paths meet their per-area floors. Client coverage (~52% statements) covers the pricing engine specifically. See A12.
☐	12.2 P1	Confirm CI gates are required, not advisory Look at Branch protection on <code>main</code> ; the four workflows in <code>.github/workflows/</code> Done when A pull request cannot merge with a failing lint, test, audit, integration or CodeQL job.
☐	12.3 P1	Verify negative and adversarial test cases exist for auth Look at <code>server/src/routes/auth.test.js</code> , <code>sso.test.js</code> , <code>app.rateLimit.test.js</code> , <code>app.passwordGate.test.js</code> , <code>middleware/requestContext.test.js</code> Done when Tests assert what must be <code>*rejected*</code> — forged tokens, revoked sessions, missing CSRF, elevated claims — not only the happy path.
☐	12.4 P2	Review the golden-master pricing tests Look at <code>client/src/screens/facultative/pricing/FacPricing.goldenMaster.test.jsx</code> ; <code>server/src/lib/pricing.goldenMaster.test.js</code> ; <code>npm run golden:fac</code> Done when Regenerating a golden master is a deliberate, reviewed act — a diff in these files must never be rubber-stamped.
☐	12.5 P2	Check for flaky and skipped tests Look at Grep for <code>.skip</code> , <code>.todo</code> , <code>only</code> ; run the suite three times Done when No test is skipped without a linked ticket; the suite is deterministic.
☐	12.6 P2	Confirm CI actions are pinned and least-privilege Look at SHA pins and <code>permissions: blocks</code> in <code>.github/workflows/*.yaml</code> Done when Every third-party action is pinned to a SHA and the default token permission is read-only.
☐	12.7 P3	Assess end-to-end coverage of the critical user journey Look at Quote to pricing to approval to bind; <code>docs/ux-testing-protocol.md</code> Done when At least one automated test walks the full revenue path against a real database.

13 Compliance, Privacy And Audit Trail

Reinsurance pricing carries commercially sensitive counterparty data. The audit trail is a product feature.

☐	13.1 P1	Verify every state-changing action is audited Look at <code>server/src/services/audit.js</code> , <code>lib/mutationAudit.js</code> , <code>services/auditActor.test.js</code> , <code>contractHistory.js</code> Done when Actor, action, entity, before/after and timestamp are captured for every mutation. Find the mutations that are not audited.
☐	13.2 P1	Confirm the audit log is tamper-evident Look at The audit table's permissions and any update/delete paths Done when Application credentials cannot update or delete audit rows; retention is defined and enforced.

☐	13.3 P1	Review the personal-data inventory against the schema Look at <code>docs/data-retention.md</code> ; <code>server/scripts/anonymize-user.js</code> (<code>gdpr:anonymize-user</code>) Done when Every column holding personal data is listed, with a lawful basis, a retention period, and a tested erasure path.
☐	13.4 P2	Verify data export and portability paths are access-controlled Look at <code>services/renewalPackExport/</code> , <code>exceljs</code> exports, <code>routes/finance.js</code> , <code>financePush.js</code> Done when An export contains only what the requesting user's mandate permits, and every export is audited.
☐	13.5 P2	Confirm AI processing is governed Look at <code>server/src/lib/aiGovernance.js</code> , <code>validateAiProviderConfig</code> in <code>app.js</code> Done when Which data leaves the estate, to which provider, under what contract and retention terms, is written down and matches the code.
☐	13.6 P2	Check vulnerability disclosure and response Look at <code>SECURITY.md</code> (Reporting a vulnerability); <code>docs/runbooks/incident-response.md</code> Done when There is a monitored contact, a stated response time, and an owner.
☐	13.7 P3	Review access-review cadence Look at <code>uw_user</code> roles and mandates; break-glass list Done when Someone reviews who holds CE/CU and break-glass access on a stated schedule, with evidence retained.

14 Frontend Review

Half the codebase, and the half where pricing is currently computed.

☐	14.1 P1	Confirm the client enforces nothing security-relevant on its own Look at Role checks in <code>client/src/screens/</code> and <code>components/</code> Done when Every client-side gate has a server-side equivalent. The client is a convenience layer, never the control.
☐	14.2 P1	Check for secrets and privileged data in the bundle Look at <code>client/dist</code> after <code>npm run build</code> ; <code>import.meta.env</code> usage Done when No API key, internal hostname or privileged configuration ships to the browser.
☐	14.3 P1	Review rendering of untrusted content Look at Grep for <code>dangerouslySetInnerHTML</code> ; imported spreadsheet and document content; <code>ExcelImportAgent.jsx</code> Done when Imported or AI-extracted content is escaped, and the CSP would contain any miss.
☐	14.4 P2	Verify error and session handling in the client Look at <code>client/src/hooks/useResource.ts</code> , <code>useToast.js</code> , <code>useScreenSave.js</code> Done when A 401 or 403 leads to a clean re-authentication; a failed save never silently discards user work.
☐	14.5 P2	Review unsaved-state handling across navigation Look at <code>useScreenSave.js</code> , <code>useEditLock.js</code> ; the recent sidebar-navigation fix (commit 80c1903) Done when Navigating away cannot silently discard edits anywhere in the wizard; this class of bug has recurred.
☐	14.6 P2	Check accessibility gating Look at <code>eslint-plugin-jsx-a11y</code> in <code>eslint.config.js</code> ; <code>npm run lint:screens</code> Done when A11y rules are errors rather than warnings on the screens, and keyboard navigation works on the critical path.
☐	14.7 P3	Assess the CSS and design-token migration Look at <code>docs/css-roadmap.md</code> ; <code>client/src/styles/</code> Done when The tokens-to-pilot plan has an owner and a next milestone, or is formally paused.

15 Documentation And Handover

The last mile of enterprise readiness: can a new engineer, or a customer's auditor, get what they need without you?

☐	15.1 P2	Verify the runbooks are current and rehearsed Look at <code>docs/runbooks/</code> (disaster-recovery, incident-response, sso-mfa-break-glass, validate-not-valid-fks) Done when Each runbook names an owner, a last-rehearsed date, and works when followed literally.
--------------------------	--------------------------	---

☐	15.2 P2	Check the deployment guide reproduces a working environment Look at <code>DEPLOYMENT.md</code> (32 KB), <code>README.md</code> , <code>CONTRIBUTING.md</code> Done when A new engineer reaches a running local stack, and an operator a running production stack, from the documents alone.
☐	15.3 P2	Confirm the API is documented for its consumers Look at <code>server/src/routes/</code> ; any OpenAPI or schema export Done when There is a generated or maintained API reference, or a written decision that the client is the only consumer.
☐	15.4 P3	Record architecture decisions Look at <code>docs/architecture.md</code> ; the decisions in items 5.3, 5.9, 6.2 and 10.1 Done when Each significant trade-off has a short written record: context, decision, consequences.
☐	15.5 P2	Close the loop on this review Look at This checklist; <code>docs/CODEBASE_AUDIT_2026-08.md</code> Done when Every P1 is closed or has a dated, signed-off acceptance; the review is re-run on a stated cadence.

A Findings Carried Forward — Verify These First

Reproduced from docs/CODEBASE_AUDIT_2026-08.md and re-checked against the current tree on 23 August 2026. Each was still present at the time of writing. Confirm, then close or accept.

☐	A1 Critical	Passwordless login is enabled on the production service render.yaml sets ALLOW_NAME_AUTH: "true" on the NODE_ENV: production service. POST /api/auth/name-login authenticates by display name alone and mints a full session with that user's real role. Unlike ALLOW_DEMO_AUTH, there is no production fence in config/env.js or startup/productionPosture.js. Chained with A2, two unauthenticated requests reach a Chief Underwriter session. Verify first, in this order: turn the flag off, add the boot fence, close A2. Checklist items 1.2, 1.5
☐	A2 High	The user directory is readable without authentication GET /auth/users is in the PUBLIC_AUTH set in server/src/routes/auth.js. Unauthenticated it still returns user_id, username, display_name, role_name and role_code for every active user — an org roster with privilege labels, useful for password spraying and phishing, and the enumeration half of A1. Checklist items 1.5
☐	A3 High	A production migration installs a shared, published password server/src/db/migrations/132_saudi_re_demo_personas.sql runs unconditionally via preDeployCommand. It sets five accounts — including one at hierarchy level 2 with approval authority — to a real script hash whose plaintext is in the file header, and clears must_change_password. Check whether it has been applied to production, then rotate. Checklist items 1.8
☐	A4 High	Document uploads fail as production is configured No Cloudinary variables and no ALLOW_LOCAL_UPLOADS in render.yaml, so assertLocalStorageAllowed() in lib/uploadStorage.js throws 503 STORAGE_NOT_DURABLE on every upload. The fail-closed design is correct; the manifest does not satisfy it. checkProductionPosture() already logs this at error level at boot. Checklist items 4.6
☐	A5 Medium	Raw error messages reach clients on 500 in production server/src/middleware/errorHandler.js:47 returns err.message for any unmapped error. Stacks are correctly withheld, but driver messages can carry schema names, paths and connection detail. Return a generic message plus the request ID. Checklist items 4.7
☐	A6 Medium	Database TLS is documented but not enforced server/src/db/pool.js passes the connection string with no ssl option, so transport security depends entirely on ?sslmode=require being present in DATABASE_URL. Nothing verifies it at boot. Checklist items 4.14
☐	A7 Medium	Rate limits are per-process with no shared store REDIS_URL is unset in render.yaml. Correct at one instance, but the ceiling multiplies by N the moment the service scales out or moves to PM2 cluster mode — most dangerous on the login limiter. Provision Redis before any scale-out. Checklist items 1.10, 10.4
☐	A8 Medium	No log-level control server/src/lib/logger.js has no LOG_LEVEL threshold, so logger.debug always writes. Impact is latent today, but there is no way to raise verbosity during an incident or suppress it under cost pressure. Checklist items 9.2

☐	A9 Low	Pricing authority originates in the browser The client derives <code>total_price</code> ; <code>lib/pricingVerifier.js</code> re-derives server-side and, with <code>PRICING_STRICT=1</code> , rejects drift with 422. That is a genuine mitigation, but the authoritative financial calculation still starts in a tamperable client. Treat as a roadmap decision, not a defect. Checklist items 5.3
☐	A10 Low	Forward-only migrations with no stated rollback policy 140 migrations, no <code>down</code> scripts. Recovery from a bad migration is restore-from-backup, which is what the DR runbook assumes. Reasonable — but it should be written down as policy. Checklist items 6.2
☐	A11 Low	Single instance, no redundancy <code>plan: starter</code> with no <code>numInstances</code> . Any deploy, crash or host event is full downtime. Defensible against the stated <code>RTO <= 4h</code> , but it should be an explicit, signed-off acceptance rather than an omission. Checklist items 10.1
☐	A12 Low	Client test coverage is thin where pricing lives Roughly 52% statements / 43% branches across ~80k lines of client code — the half where pricing is computed. Server coverage is materially better. The ratchet policy is sound; the client floor needs raising over time. Checklist items 12.1

B Sign-Off

Complete once every P1 above is closed or carries a written acceptance.

Area	P1 open	P2 open	Accepted risks	Reviewed by	Date
0 Set Up And Reproduce The Baseline					
1 Authentication And Login					
2 Sessions, Tokens And Cookies					
3 Authorization, Roles And Mandates					
4 Application Security Controls					
5 Architecture And Code Structure					
6 Data Layer, Migrations And Integrity					
7 Secrets And Configuration					
8 Dependencies And Supply Chain					
9 Observability And Operations					
10 Reliability, Availability And Recovery					
11 Performance And Scale					
12 Testing And Delivery Pipeline					
13 Compliance, Privacy And Audit Trail					
14 Frontend Review					
15 Documentation And Handover					
Overall — cleared for enterprise deployment					

A signature here states that the reviewer has personally verified each ticked item against the running system, and that every open risk is recorded, owned and accepted by a named person. Re-run this review at each major release, and after any change to authentication, authorization, or the production manifest.